Université Paris Cité

ÉCOLE D INGÉNIEUR DENIS DIDEROT

Projet – Circuits et architecture :
Microprocesseur LC3

EIDD 2A – SIE

BENLAFQIH AHMED
BERKANI Oumaima

2024 – 2025

1- Introduction :

Ce projet avait pour objectif principal de modifier une Unité Arithmétique et Logique (ALU) d'un microprocesseur basé sur l'architecture LC-3 pour y ajouter des instructions et des fonctionnalités avancées. Parmi les nouvelles instructions implémentées figurent **Bit Scan Forward (BSF)**, **Bit Scan Backward (BSB)**, ainsi que des opérations liées à la gestion des données en mémoire comme **LD** (Load), **ST** (Store), et **LEA** (Load Effective Address). L'objectif était de modifier exclusivement le module ALU et les composants essentiels pour garantir que ces instructions soient correctement interprétées, exécutées, et que les drapeaux de statut (NZP) soient mis à jour de manière cohérente.

Ce rapport détaille les étapes suivies pour concevoir et implémenter ces instructions, les changements apportés au circuit, les programmes de test écrits pour valider les fonctionnalités ajoutées, et une analyse des résultats obtenus. L'ensemble du travail a été réalisé en respectant les contraintes de faible coût en portes logiques et de profondeur de circuit.

2- Conception du circuit : Changement Apporté

2-1- BSF/BSB :

Ce circuit est conçu pour exécuter les instructions BSF (Bit Scan Forward) et BSB (Bit Scan Backward) en localisant le premier bit à `1` dans une entrée binaire de 16 bits. Il utilise une entrée principale appelée **Input** et un bit de contrôle `IR(bit5)` pour déterminer le mode d'opération : BSF (analyse LSB → MSB) ou BSB (analyse MSB → LSB).
Pour gérer la direction, un inverseur de bits est connecté à un multiplexeur (MUX) qui sélectionne soit l'entrée brute, soit son inversion, en fonction de `IR(bit5)`. La sortie est ensuite analysée par un **encodeur prioritaire** qui identifie la position du premier bit à `1` et la convertit en un indice de 4 bits. Ce résultat est ensuite étendu à 16 bits pour s'adapter au reste du circuit.

Le circuit inclut également une logique de comparaison entre deux signaux internes, `b in` et `b out`, pour valider les résultats avant qu'ils ne soient transmis à un registre de destination via un second MUX. En résumé, ce circuit combine l'inversion de bits, l'encodage prioritaire et des multiplexeurs pour exécuter efficacement BSF et BSB tout en garantissant la fiabilité des résultats.

2-2- Encodeur Prioritaire :

Un encodeur prioritaire a été ajouté pour analyser les 16 bits du registre source (`SR`). Il identifie directement l'indice du premier bit à `1`.

2-3- Inverseur de Bits :

Pour BSB, un inverseur de bits réordonne les bits du registre source (à savoir `SR[15]` → `SRRev[0]`, etc.), permettant une recherche MSB vers LSB.

2-4- Decodeur IR :

Ce circuit est une unité de contrôle qui décode les instructions stockées dans le registre **IR** pour générer des signaux de contrôle tels que **Jump**, **Arith**, **WriteReg**, **Load**, et **Store**. Les bits d'instruction **[15:14]** et **[12:13]** sont utilisés pour déterminer le type d'opération.

Le **MUX** traite les bits **[15:14]** pour activer des signaux comme **Jump** ou **Arith**, tandis que le **DMX** utilise les bits **[12:13]** pour gérer des instructions liées à la mémoire, comme **Load** et **Store**. Ces signaux sont combinés par des portes logiques pour produire les sorties finales. Ce circuit joue un rôle essentiel en orientant les différentes unités du processeur selon l'instruction en cours d'exécution.

2-5- WriteVal :

Ce circuit décide de la valeur à charger dans le registre cible (**RegIN**) en fonction de l'instruction en cours. Si l'instruction est **JSR**, il utilise le compteur de programme (**PC**). Pour **LEA**, il calcule et utilise **PC + SEXT(Offset9)**, où Offset9 est une extension signée. Si l'instruction est **Load**, il utilise la valeur de **MemOUT**, et dans tous les autres cas, il sélectionne la sortie de l'ALU (**ALURes**). Le circuit utilise des portes logiques pour identifier le type d'instruction à partir des bits du registre d'instruction (**IR**) et contrôle un multiplexeur qui choisit la source appropriée pour alimenter **RegIN**, garantissant ainsi que chaque instruction reçoit les données nécessaires.

2-6- NZP :

Ce circuit détermine si une mise à jour des drapeaux NZP (Negative, Zero, Positive) doit être effectuée en fonction du résultat courant (**RES**) et de l'instruction en cours (**IR**).

Si l'instruction nécessite une mise à jour (contrôlée par le signal **WriteReg**), le circuit utilise un multiplexeur (**MUX**) pour sélectionner le résultat approprié parmi les différentes comparaisons effectuées sur **RES** (par exemple, **>** ou **<**). Le résultat de cette sélection est combiné avec un test logique sur **IR[11:9]** (ou un autre champ pertinent), via des portes AND et OR, pour générer le signal final **TestNZP**, qui indique si les drapeaux NZP doivent être modifiés. Ce circuit garantit que seuls les résultats valides et pertinents sont utilisés pour mettre à jour les drapeaux NZP.

3- Programmes de Test et Validation :

Pour valider le bon fonctionnement des instructions ajoutées, plusieurs programmes de test en assembleur LC-3 ont été écrits. Voici quelques exemples représentatifs.

3-1- Test de BSF :

Ce programme teste la recherche du premier bit à `1` à partir du LSB.

```
.ORIG x3000
LD R1, DATA ; Charger R1 avec la donnée
BSF R2, R1 ; Trouver le premier bit à 1 dans R1
HALT
DATA .FILL x8002 ; Donnée à analyser (1000 0000 0000 0010)
.END
```

Résultat attendu : `R2 = 1` (le premier bit à `1` est à l'indice 1).

3-2- Test de BSB :

Ce programme teste la recherche du premier bit à `1` à partir du MSB.

```
.ORIG x3000
LD R1, DATA ; Charger R1 avec la donnée
BSB R2, R1 ; Trouver le premier bit à 1 en partant du MSB
HALT
DATA .FILL x8002 ; Donnée à analyser (1000 0000 0000 0010)
.END
```

Résultat attendu : `R2 = 15` (le premier bit à `1` en partant du MSB est à l'indice 15).

3-3- Test de LD et LEA :

```
.ORIG x3000
LEA R1, LABEL ; Charger l'adresse de LABEL dans R1
LD R2, LABEL ; Charger la valeur de LABEL dans R2
HALT
LABEL .FILL x1234 ; Valeur stockée en mémoire
.END
```

Résultat attendu : `R1` contient l'adresse de `LABEL`, et `R2` contient la valeur `x1234`.

Résultats et Analyse :

Les tests réalisés ont confirmé le bon fonctionnement des instructions ajoutées. Les résultats produits par BSF et BSB correspondent à ceux attendus dans tous les cas testés. Les instructions

LD, ST, et LEA fonctionnent également correctement, avec des accès mémoire gérés de manière fiable. Les drapeaux NZP sont mis à jour de manière cohérente pour chaque instruction.

Cependant, quelques défis ont été rencontrés. Par exemple, l'ajout de l'inverseur de bits pour BSB a initialement provoqué des erreurs de connexion, résolues en simplifiant le câblage. De plus, le multiplexeur pour basculer entre BSF et BSB a requis une configuration précise pour éviter les conflits entre signaux.

4- Conclusion:

Ce projet a permis d'étendre les capacités du processeur LC-3 en ajoutant des instructions avancées comme BSF et BSB, tout en enrichissant son jeu d'instructions avec des opérations essentielles comme LD, ST, et LEA. Les modifications apportées au circuit ont respecté les contraintes imposées, en garantissant un faible coût en portes logiques et une mise à jour correcte des drapeaux NZP.

Pour aller plus loin, des optimisations supplémentaires pourraient être envisagées, notamment pour réduire la profondeur du circuit et améliorer la lisibilité globale. Un système de test automatisé pourrait également être intégré pour valider l'ensemble des instructions de manière systématique.